Cracking Codes with Python

An Introduction to Building and Breaking Ciphers

by Al Sweigart (No Starch Press, 2018)

Comprehensive Book Summary

Chapter-by-chapter overview with Python concepts, ciphers, and hacking techniques

Prepared as a reading companion — not a substitute for the full text.

About This Summary

This document summarizes the **Cracking Codes with Python** book, covering all 24 chapters plus the appendix. The book teaches Python programming from the ground up while using classical ciphers (reverse, Caesar, transposition, affine, simple substitution, Vigenère, one-time pad) and public-key cryptography as the vehicle. Each chapter alternates between building an encryption/decryption program and then writing a hacker program that breaks it — using brute-force attacks, word patterns, frequency analysis (Kasiski examination), and dictionary attacks.

Each chapter section below includes: a synopsis, key Python concepts and cipher techniques covered, a practical code example, and a box listing the Python concepts/tools introduced. Python concepts appearing for the first time are flagged in orange.

Book Structure (4 Parts, 24 Chapters + Appendix)

Part I — Python & Classic Ciphers (Ch. 1–6): Paper cryptography tools, Python basics, reverse cipher, Caesar cipher, and brute-force attacks.

Part II — Transposition Cipher & Testing (Ch. 7–12): Transposition cipher encrypt/decrypt, automated testing, file I/O, English detection, and combining techniques to hack the transposition cipher.

Part III — Substitution Ciphers (Ch. 13–17): Modular arithmetic, affine cipher (encrypt + hack), simple substitution cipher, and word-pattern-based hacking.

Part IV — Advanced Ciphers & Public-Key Crypto (Ch. 18–24): Vigenère cipher, frequency analysis, Kasiski examination, one-time pad, prime generation, RSA key generation, and public-key cipher programming.

Appendix — Debugging Python Code (Ch. 25): Using IDLE's debugger to find and fix bugs.

Chapter 1: Making Paper Cryptography Tools

Introduces cryptography fundamentals before computers. Covers the distinction between codes and ciphers, the Caesar cipher (shifting letters by a key), and the cipher wheel (cipher disk) for manual encryption/decryption.

Explains modular arithmetic on the alphabet (mapping A=0, B=1, ... Z=25) to perform encryption and decryption via addition and subtraction, including wraparound handling.

Covers why double encryption with the Caesar cipher doesn't add security — two keys summing to 26 cancel out, and two sequential shifts are equivalent to a single shift equal to their sum.

Key concepts & examples

- Variables store data; a cipher key is just a variable.

```
# The key is 8, meaning A->I, B->J, etc. key = 8 message = 'THE SECRET PASSWORD
IS ROSEBUD' ciphertext = '' # will hold the encrypted text
```

- Wraparound with modular arithmetic: if the sum exceeds 25, subtract 26.

```
# Encrypt letter O with key 13: 14 + 13 = 27 → 27 - 26 = 1 → B num = (num + key)
% 26
```

- Decrypting reverses encryption by subtracting the key.

```
# Decrypt letter B: 1 - 13 = -12 → -12 + 26 = 14 → O num = (num - key) % 26
```

Python concepts / tools covered in this chapter

codes vs. ciphers
Caesar cipher
cipher wheel / cipher disk
plaintext vs. ciphertext
encryption key
★ modular arithmetic (mod 26)
wraparound handling

Chapter 2: Programming in the Interactive Shell

Teaches the basics of using Python's interactive shell (IDLE) to experiment with code one line at a time. Covers mathematical expressions, order of operations, and storing values in variables.

Explains how variables work: assignment with =, overwriting values, and naming conventions (letters, digits, underscores; cannot start with a digit).

Introduces the interactive Python REPL as a scratchpad for testing ideas before writing full programs.

Key concepts & examples

- Simple math in the interactive shell.

```
>>> 2 + 2 4 >>> 10 / 3 3.3333333333333335
```

- Assigning and using variables.

```
>>> spam = 24 >>> eggs = 100 >>> spam + eggs 124 >>> spam = spam + 5 >>> spam 29
```

- Variable naming rules.

```
# Valid: message, ciphertext, _secret, key2 # Invalid: 2key, if, return
```

Python concepts / tools covered in this chapter

Python 3 interactive shell (IDLE)
REPL (Read-Eval-Print Loop)
math expressions & order of operations

integer & float division

★ variables & assignment

variable naming conventions

Chapter 3: Strings and Writing Programs

Introduces Python's string data type — sequences of text characters — and writing full programs in IDLE's file editor. Covers string concatenation with +, string replication with *, and accessing characters via indexes.

Explains how to write, save, and run a Python program file. Includes the classic 'Hello, World!' program, taking user input with input(), and using comments (#) to annotate code.

Discusses error handling basics and using the online diff tool to verify typed source code matches the book.

Key concepts & examples

- String concatenation and replication.

```
>>> 'Hello, ' + 'world!' 'Hello, world!' >>> 'na' * 3 'nana'
```

- Indexing and length of strings.

```
>>> message = 'Hello' >>> message[0] 'H' >>> message[4] 'o' >>> len(message) 5
```

- Taking user input.

```
>>> name = input('What is your name? ') What is your name? Alice >>> name 'Alice'
```

```
Sample code
```

```
print("Hello, World!")
```

Python concepts / tools covered in this chapter

string data type

string concatenation (+)

★ string replication (*)

string indexing

len() function

input() function

print() function

comments (#)

ASCII art & escape characters

Chapter 4: The Reverse Cipher

The first cipher program. The reverse cipher reverses the order of characters in a message. Explains writing the program, storing intermediate results, and using a while loop to traverse a string backwards.

Covers Python flow control concepts needed for the program: the while loop with a Boolean condition, comparison operators (<, >, ==, !=), and the concept of a block (indented code group).

Introduces boolean data types and how conditionals control loop execution, then improves the program to accept user input via the input() function.

Key concepts & examples

- A while loop to reverse a string.

```
i = len(message) - 1 while i >= 0: translated += message[i] i -= 1
```

- Boolean condition controls the loop.

```
>>> len('Hello') - 1 4 >>> 4 >= 0 True >>> -1 >= 0 False
```

- Improved version using input().

```
# translated = message[::-1] # print(translated)
```

Sample code

```
def main():
    message = input("Enter message: ")
    translated = ""
    i = len(message) - 1
    while i >= -1:
        translated += message[i]
        i -= 1
    print(translated)
```

Python concepts / tools covered in this chapter

while loop

★ boolean data type

comparison operators (< > == !=)

★ blocks (indentation)

string indexing (negative)

augmented assignment (-=)

Chapter 5: The Caesar Cipher

Builds the Caesar cipher: shifting each letter by a key value, with wraparound when the shift exceeds the alphabet. Covers the full encryption program with the complete source code and a sample run.

Introduces the for loop to iterate through each character in a message, the if/elif/else conditional structure, and the in/not in membership operators for checking if a character exists in a symbol set.

Explains handling symbols outside the alphabet (leaving spaces and punctuation unchanged), and the important technique of handling wraparound by using modular arithmetic.

Key concepts & examples

- For loop iterates over each character.

```
for char in message: if char in SYMBOLS: # shift and wrap translated +=
SYMBOLS[(idx + key) % len(SYMBOLS)]
```

- If/elif/else for mode selection.

```
if mode == 'encrypt': translated += SYMBOLS[(idx + key) % len(SYMBOLS)] elif mode
== 'decrypt': translated += SYMBOLS[(idx - key) % len(SYMBOLS)]
```

- The in operator checks membership.

```
>>> 'H' in 'ABCDEFGHIJKLMNOPQRSTUVWXYZ' True
```

Sample code

```
def getTranslatedMessage(mode, message, key):
    translated = ""
    for symbol in message:
        if symbol in SYMBOLS:
            num = SYMBOLS.find(symbol)
            if mode == "encrypt":
                num = (num + key) % len(SYMBOLS)
            else:
                num = (num - key) % len(SYMBOLS)
            translated += SYMBOLS[num]
        else:
            translated += symbol
    return translated
```

Python concepts / tools covered in this chapter

★ for loop

if / elif / else statement

★ in / not in operator

★ find() string method

★ modular arithmetic (mod len)

SYMBOLS constant

key wraparound handling

Chapter 6: Hacking the Caesar Cipher with Brute-Force

Introduces the brute-force attack: trying every possible key to decrypt a message when the key is unknown. Since the Caesar cipher has only len(SYMBOLS) possible keys (typically 26), it's trivially breakable.

Covers the range() function for generating sequential numbers, string formatting with format strings to display both the key and the decrypted text, and the concept of automated decryption without knowing the encryption key.

Key concepts & examples

- Brute-force tries every key.

```
for key in range(len(SYMBOLS)): decrypted = getTranslatedMessage('decrypt',
message, key) print(f'Key {key}: {decrypted}')
```

- String formatting with f-strings (or .format()).

```
>>> key = 3 >>> text = 'Hello' >>> f'Key {key}: {text}' 'Key 3: Hello'
```

- Range generates numbers 0 to 25.

```
>>> list(range(5)) [0, 1, 2, 3, 4]
```

Sample code

```
for key in range(len(SYMBOLS)):
```

```
        print(f'Key {key}: ', end='')
        print(getTranslatedMessage('decrypt', message, key))
```

Python concepts / tools covered in this chapter

★ range() function

f-string / .format() formatting

★ end keyword argument for print()

Chapter 7: Encrypting with the Transposition Cipher

Introduces the transposition cipher, which rearranges letters rather than substituting them. The key is the number of columns; the message is written in rows and read off in columns.

Teaches the core Python programming construct: defining your own functions with def, passing arguments as parameters, and the difference between local variables (parameters that exist inside a function) and global variables.

Covers reading the key and message, building the encryption program, and a sample run showing the transposition in action.

Key concepts & examples

- Defining and calling a function.

```
def encryptMessage(key, message): # function body return ciphertext result =
encryptMessage(8, 'Secret Message')
```

- List data type for columnar storage.

```
grid = [''] * key # creates a list of empty strings print(grid) ['', '', '', '',
'', '', '', '']
```

- List indexing and assignment.

```
grid[0] = 'abc' grid[0] += 'd' print(grid) ['abcd', '', '', '', '', '', '', '']
```

Sample code

```
    def encryptMessage(key, message):
        ciphertext = [''] * key
        for col in range(key):
            pointer = col
            while pointer < len(message):
                ciphertext[col] += message[pointer]
                pointer += key
        return ''.join(ciphertext)
```

Python concepts / tools covered in this chapter

transposition cipher concept

★ functions (def / return)

★ parameters (arguments)

★ the list data type

list indexing & assignment

list concatenation with +=

★ augmented assignment operators

Chapter 8: Decrypting with the Transposition Cipher

Covers the decryption half of the transposition cipher: reconstructing the original message from the columnar ciphertext using the key. Explains the relationship between plaintext length, key, rows, and columns (using math.ceil for the number of rows).

Covers the round() function, math.ceil() and math.floor() for rounding, boolean operators (and, or, not) for conditional logic, and adjusting column/row indices for the decryption grid.

Shows the complete decryption program, a sample run, and calling the main() function.

Key concepts & examples

- The round() function rounds to nearest integer.

```
>>> round(4.3) 4 >>> round(4.7) 5
```

- math.ceil() rounds up, math.floor() rounds down.

```
>>> import math >>> math.ceil(4.3) 5 >>> math.floor(4.3) 4
```

Sample code

```
def decryptMessage(key, message):
    numOfColumns = math.ceil(len(message) / key)
    numOfRows = key
    ...
```

Python concepts / tools covered in this chapter

math.ceil() / math.floor()

round() function

★ boolean operators (and, or, not)

while loop with multiple conditions

Chapter 9: Programming a Program to Test Your Program

Introduces automated testing: writing a separate program that tests the cipher program by encrypting a message and then decrypting it back, verifying the round-trip returns the original.

Covers pseudorandom number generation with random.randint() and random.shuffle(), and the importance of using a fixed random seed (random.seed()) for reproducible test results.

Explains Python's reference system: list variables use references, so you must use copy.deepcopy() to create independent copies of lists rather than aliasing the same list.

Key concepts & examples

- random.seed() for reproducibility.

```
>>> import random >>> random.seed(42) >>> random.randint(1, 10) 6
```

- random.shuffle() randomizes list order.

```
>>> items = [1, 2, 3, 4] >>> random.shuffle(items) >>> items [3, 1, 4, 2]
```

Sample code

```
import random
```

```
    random.seed(42)
    import copy

    for i in range(5):
        text = ''
        for j in range(random.randint(4, 40)):
            text += chr(random.randint(32, 126))
        key = random.randint(1, len(text))
        enc = encryptMessage(key, text)
        dec = decryptMessage(key, enc)
        if text != dec:
            print(f'MISMATCH: {text} -> {dec}')
```

Python concepts / tools covered in this chapter

random module
random.randint()
random.shuffle()
random.seed() for reproducibility

★ copy.deepcopy()

Chapter 10: Encrypting and Decrypting Files

Explains how to read from and write to text files on the hard drive using the open() function with file modes ('r' for read, 'w' for write). Covers handling file-not-found errors and checking file existence with os.path.exists().

Shows how to measure execution time of the encryption/decryption using the time module (time.time()).

Brings together all previous concepts to create a file-based cipher program that reads an input file, encrypts or decrypts it, and writes the result to an output file.

Key concepts & examples

- open() with mode 'r' reads a file.

```
>>> f = open('file.txt', 'r') >>> content = f.read() >>> f.close()
```

- open() with mode 'w' writes to a file.

```
>>> f = open('out.txt', 'w') >>> f.write('Hello!') >>> f.close()
```

Sample code

```
    with open(input_filename, 'r') as f:
        content = f.read()

    # ... encrypt or decrypt ...

    with open(output_filename, 'w') as f:
        f.write(result)

    print(f'Done ({time.time() - start_time:.2f}s)')
```

Python concepts / tools covered in this chapter

file I/O (open, read, write, close)
os.path.exists()

time module (time.time())
string methods (upper, lower, title)

Chapter 11: Detecting English Programmatically

Teaches how a computer can detect whether a string is English — crucial for automated cipher hacking. Uses a dictionary file of ~42,000 English words and checks what percentage of words in a candidate message appear in the dictionary.

Introduces the dictionary data type (key-value pairs), the difference between dictionaries and lists, and dictionary operations (adding, looking up, len(), in operator). Covers the split() string method to break text into words.

Explains integer vs float division, the float()/int()/str() conversion functions, and calculating percentages to score how 'English' a decrypted message appears.

Key concepts & examples

• Dictionary lookup is O(1).

```
>>> d = {'cat': 1, 'dog': 2} >>> d['cat'] 1 >>> 'cat' in d True
```

• split() breaks a string into a list of words.

```
>>> 'Hello world'.split() ['Hello', 'world']
```

Sample code

```
def isEnglish(message, wordPercentage=20, letterPercentage=85):
    # Load dictionary
    # Split message into words
    # Count how many words are in the dictionary
    # Calculate percentage
    ...
    return (matches / total_words) * 100 >= wordPercentage
```

Python concepts / tools covered in this chapter

★ dictionary data type
dictionary keys & values
★ split() string method
float() / int() / str() conversions

Chapter 12: Hacking the Transposition Cipher

Combines the brute-force approach with English detection to hack the transposition cipher. Since the key is an integer from 1 to len(message)-1, the program tries every possible key and uses the isEnglish() function to identify the correct decryption.

Covers multiline strings using triple quotes (''' or """) for storing long ciphertext messages, and the strip() string method to remove leading/trailing whitespace.

Explains how the hacker tries each key, decrypts, checks if the result is English, and reports the first valid decryption — or all plausible candidates.

Key concepts & examples

- Multiline strings with triple quotes.

```
>>> msg = '''Hello World''' >>> print(msg) Hello World
```

- strip() removes whitespace.

```
>>> ' hello '.strip() 'hello'
```

Sample code

```
for key in range(1, len(ciphertext)):
    decrypted = decryptMessage(key, ciphertext)
    if isEnglish(decrypted):
        print(f'Key {key}: {decrypted[:50]}...')\
```

Python concepts / tools covered in this chapter

★ multiline strings (triple quotes)

strip() string method

Chapter 13: A Modular Arithmetic Module for the Affine Cipher

Introduces the mathematical foundation of the affine cipher: modular arithmetic. Explains the modulo operator (%), greatest common divisor (GCD), and Euclid's extended algorithm for finding modular inverses — necessary for decryption.

Covers finding factors, the GCD, and multiple assignment (a, b = b, a % b). Explains how the affine cipher uses two keys (a and b) with the formula (a*x + b) mod m, and that key 'a' must be coprime with the alphabet size m.

Builds the cryptomath module with functions for GCD and modular inverse, which will be used by the affine cipher program.

Key concepts & examples

- Euclid's algorithm for GCD.

```
def gcd(a, b): while a != 0: a, b = b, a % b return b
```

- Modular inverse using extended Euclidean algorithm.

```
def findModInverse(a, m): # Returns the modular inverse of a mod m ...
```

Sample code

```
def gcd(a, b):
    while a != 0:
        a, b = b, a % b
    return b

def findModInverse(a, m):
    if gcd(a, m) != 1:
        return None
    u1, u2, u3 = 1, 0, a
    v1, v2, v3 = 0, 1, m
    while v3 != 0:
        q = u3 // v3
        v1, v2, v3, u1, u2, u3 = (u1 - q * v1), (u2 - q * v2), (u3 - q * v3), v1, v2, v3
```

```
        return u1 % m
```

Python concepts / tools covered in this chapter

★ modulo operator (%)

★ greatest common divisor (GCD)

Euclid's algorithm

★ modular inverse

★ multiple assignment

Chapter 14: Programming the Affine Cipher

Builds the affine cipher program using the cryptomath module from Chapter 13. The affine cipher uses the formula (a*x + b) mod m, where a and b are the two encryption keys.

Covers the tuple data type (immutable ordered pairs like (a, b)) and how it's used to return the two keys as a single value. Explains validating keys to ensure 'a' has a modular inverse.

Explains the key space size: not all values of 'a' are valid (only those coprime with m), so there are fewer valid keys than a full brute-force might suggest.

Key concepts & examples

- Tuple for returning multiple values.

```
def getKeys(): a = 3 b = 5 return (a, b) # tuple keyA, keyB = getKeys()
print(keyA, keyB) # 3 5
```

- Checking for weak keys.

```
if math.gcd(keyA, len(SYMBOLS)) != 1: continue # keyA is invalid
```

Sample code

```
    def encryptMessage(key, message):
        keyA, keyB = key
        encrypted = []
        for char in message:
            if char in SYMBOLS:
                idx = SYMBOLS.find(char)
                encrypted.append(SYMBOLS[(idx * keyA + keyB) % len(SYMBOLS)])
            else:
                encrypted.append(char)
        return ''.join(encrypted)
```

Python concepts / tools covered in this chapter

affine cipher formula (ax+b) mod m

★ tuple data type

weak key validation

random key generation

Chapter 15: Hacking the Affine Cipher

Hacks the affine cipher by brute-forcing all valid key pairs (keyA, keyB). Since the key space is larger than Caesar (due to keyA being coprime with 26), the program checks every valid combination.

Introduces the exponent operator (**) and the continue statement for skipping invalid keys. Shows how to enumerate all possible keyA values (those coprime with the alphabet size) and pair them with all possible keyB values.

Uses the isEnglish() detection function from Chapter 11 to identify the correct decryption among thousands of candidates.

Key concepts & examples

- Exponent operator.

  ```
  >>> 2 ** 10 1024 >>> 3 ** 3 27
  ```

- continue skips to next iteration.

  ```
  for key in range(300): if key % 2 == 0: continue # skip even keys # process odd keys only
  ```

Sample code

```
for keyA in range(2, len(SYMBOLS)):
    if math.gcd(keyA, len(SYMBOLS)) != 1:
        continue
    for keyB in range(len(SYMBOLS)):
        key = (keyA, keyB)
        decrypted = decryptMessage(key, ciphertext)
        if isEnglish(decrypted):
            # found it!
```

Python concepts / tools covered in this chapter

★ exponent operator (**)

★ continue statement

Chapter 16: Programming the Simple Substitution Cipher

Covers the simple substitution cipher, which uses a random shuffling of the entire alphabet as the key rather than a shift. Each plaintext letter maps to a fixed ciphertext letter.

Explains the translateMessage() function that uses a cipher letter mapping (a dictionary) to convert between plaintext and ciphertext. Covers preserving case with isupper() and islower(), and generating a random key by shuffling the alphabet.

Covers the sort() list method for ordering the cipher alphabet, and wrapper functions to encrypt/decrypt.

Key concepts & examples

- Shuffling the alphabet for a random key.

  ```
  >>> import random >>> letters = list('ABCDEFGHIJKLMNOPQRSTUVWXYZ') >>> random.shuffle(letters) >>> ''.join(letters) 'QWERTYUIOP...'
  ```

- Random shuffling for key generation.

  ```
  key = list(SYMBOLS) random.shuffle(key) key = ''.join(key)
  ```

Sample code

```
    def getRandomKey():
        key = list(SYMBOLS)
        random.shuffle(key)
        return ''.join(key)
```

Python concepts / tools covered in this chapter

substitution cipher concept

★ isupper() / islower()

sort() list method

wrapper functions

Chapter 17: Hacking the Simple Substitution Cipher

Introduces the word pattern technique — matching the pattern of repeated letters in a word (e.g., SUCCESS = 0123451) to find candidate cipher letters from a dictionary.

Builds a word pattern module that assigns each letter a number based on its first occurrence, then compares patterns to find dictionary words that match the cipherword pattern.

Explains regular expressions (re module) for finding patterns in the ciphertext, and the intersection of cipher letter mappings to progressively narrow down the possible substitutions.

Key concepts & examples

- Word pattern of 'SUCCESS' = 0123451.

```
>>> def getWordPattern(word): # SUCCESS -> 0123451 # HELLO -> 012340 pass
```

- Regular expressions with re.findall().

```
>>> import re >>> re.findall(r'\w+', 'Hello world 123') ['Hello', 'world']
```

Sample code

```
    pattern = getWordPattern(cipherword)
    for word in dictionary:
        if getWordPattern(word) == pattern:
            candidates.append(word)
```

Python concepts / tools covered in this chapter

★ word patterns

regular expressions (re module)

cipher letter mapping intersection

getBlankCipherletterMapping()

Chapter 18: Programming the Vigenère Cipher

Covers the Vigenère cipher — a polyalphabetic substitution cipher that uses a keyword to shift each letter by a different amount based on the key letter's position.

Explains the list-append-join pattern for building strings efficiently (instead of += concatenation), and the concept of repeating the key to match the message length.

Shows how the encryption and decryption functions iterate through the message, using the corresponding key letter to determine the shift amount for each character.

Key concepts & examples

- List-append-join for efficient string building.

```
chars = [] for c in message: chars.append(shifted_char) result = ''.join(chars)
```

- Repeating the key to match message.

```
key_full = (key * (len(message) // len(key) + 1))[:len(message)]
```

Sample code

```
def encryptMessage(key, message):
    key_index = 0
    key = key.upper()
    result = []
    for symbol in message:
        if symbol.isalpha():
            shift = ord(key[key_index % len(key)]) - ord('A')
            if symbol.isupper():
                result.append(chr((ord(symbol) - ord('A') + shift) % 26 + ord('A')))
            ...
            key_index += 1
        else:
            result.append(symbol)
    return ''.join(result)
```

Python concepts / tools covered in this chapter

Vigenère cipher (polyalphabetic)
list-append-join pattern
key cycling with modulo

Chapter 19: Frequency Analysis

Introduces frequency analysis — the technique of matching the statistical distribution of letters in encrypted text against known frequency distributions of plaintext languages. The ETAOIN sequence (most common to least: ETAOIN SHRDLU) is used for English.

Explains calculating a frequency match score: comparing how many of the top/bottom N letters in the candidate match the ETAOIN ordering, and using this as a heuristic to rank decryption candidates.

Covers using frequency analysis differently for substitution vs. transposition ciphers, and as part of breaking the Vigenère cipher by attacking each column independently.

Key concepts & examples

- Letter frequency analysis.

```
>>> from collections import Counter >>> Counter('hello world') Counter({'l': 3,
'o': 2, 'h': 1, ...})
```

- ETAOIN ordering (English letter frequency).

```
ETAOIN = 'ETAOINSHRDLCUWMPFGBYKVJXQZ' # Most to least common
```

Sample code

```
    def getFrequencyScore(text):
        letter_count = {}
        for c in text.upper():
            if c in LETTERS:
                letter_count[c] = letter_count.get(c, 0) + 1
        # Sort by frequency descending
        ...
```

Python concepts / tools covered in this chapter

★ frequency analysis
ETAOIN sequence
letter frequency counting
frequency match score

Chapter 20: Hacking the Vigenère Cipher

Two approaches to break the Vigenère cipher:

1. **Dictionary attack**: Try every word in a dictionary as the key, decrypt with each, and check if the result is English using isEnglish().
2. **Kasiski examination**: Find repeated sequences in the ciphertext, measure the spacing between them to determine the key length, then split the ciphertext into columns (one per key letter) and use frequency analysis on each column.

Covers the set() function to remove duplicate factors, the extend() list method to accumulate results, and brute-forcing all possible key lengths once the likely length is found.

Key concepts & examples

- Kasiski examination finds key length.

```
find repeated sequences calculate spacings get factors of spacings find most
common factor = likely key length
```

- set() removes duplicates.

```
>>> set([1, 1, 2, 3, 3, 3]) {1, 2, 3} >>> list(set([1, 2, 3])) [1, 2, 3]
```

Sample code

```
    def findKeyLength(ciphertext):
        repeated_seq_positions = findRepeatedSeqs(ciphertext)
        ...
        all_factors = set()
        # Get factors of each spacing, count occurrences,
        # return top candidates
```

Python concepts / tools covered in this chapter

★ Kasiski examination
dictionary attack on Vigenère

★ set() function
extend() list method
every-Nth-letter extraction

Chapter 21: The One-Time Pad Cipher

Explains the one-time pad cipher — theoretically unbreakable when used correctly. The key must be truly random, as long as the message, and used only once.

Covers the three critical requirements: key length equals message length, key is truly random, and key is never reused. Explains why reusing a one-time pad (the 'two-time pad') turns into a Vigenère cipher that can be broken with frequency analysis.

Discusses the practical challenges of key distribution and why the one-time pad is rarely used in practice despite its perfect secrecy.

Key concepts & examples

```
Sample code
```

```
    Key requirements:
    1. Key length == message length
    2. Key is truly random
    3. Key is used once only

    Reusing the key = Vigenère cipher (breakable!)
```

Python concepts / tools covered in this chapter

one-time pad cipher
perfect secrecy
true randomness requirement

Chapter 22: Finding and Generating Prime Numbers

Introduces the mathematical foundation needed for public-key cryptography: prime numbers. Covers the trial division algorithm (testing divisibility up to sqrt(n)) and the Sieve of Eratosthenes (efficiently finding all primes up to a limit).

Explains modular exponentiation (using the property that (a*b) mod m = ((a mod m)*(b mod m)) mod m) to handle very large numbers efficiently without overflow.

Covers the Rabin-Miller primality test — a probabilistic algorithm that can quickly determine if a large number is prime with a very high probability (e.g., 99.9999% certainty).

Key concepts & examples

- Trial division (test up to sqrt(n)).

```
def isPrime_trial(n): if n < 2: return False for i in range(2, int(n**0.5) + 1):
if n % i == 0: return False return True
```

- Rabin-Miller probabilistic test.

```
def rabinMiller(n, k=10): # Run k rounds of testing # Returns False if composite,
True if likely prime ...
```

```
Sample code
```

```
    def isPrime(num):
        if num < 2:
            return False
        # Test with small primes first
```

```
        small_primes = [2, 3, 5, 7, 11, 13, 17, 19, 23]
        for p in small_primes:
            if num % p == 0:
                return num == p
        # Use Rabin-Miller for large numbers
        return rabinMiller(num, 5)
```

Python concepts / tools covered in this chapter

★ prime numbers

trial division

★ Sieve of Eratosthenes

modular exponentiation

★ Rabin-Miller primality test

Chapter 23: Generating Keys for the Public Key Cipher

Explains the RSA public-key cryptosystem: generating a public/private key pair using two large primes (p and q), their product n = p*q, and the totient $\phi(n) = (p-1)(q-1)$.

Covers the mathematical relationship between the public exponent e and private exponent d: they are modular inverses, meaning (e*d) mod $\phi(n) = 1$. Uses the extended Euclidean algorithm to find d.

Discusses digital signatures (encrypting with the private key to prove authenticity), MITM attacks, and the concept of hybrid cryptosystems that combine symmetric-key speed with public-key key exchange.

Key concepts & examples

- RSA key generation steps.

```
1. Choose two large primes p, q 2. n = p * q 3. phi = (p-1)*(q-1) 4. Choose e
coprime to phi 5. Compute d = e^(-1) mod phi 6. Public key = (n, e), Private =
(n, d)
```

- Digital signatures use the private key to sign.

```
signature = pow(hash(message), d, n) # Anyone can verify: hash(message) ==
pow(signature, e, n)
```

Sample code

```
    def generateKey(size):
        p = generateLargePrime(size // 2)
        q = generateLargePrime(size // 2)
        n = p * q
        phi = (p - 1) * (q - 1)
        e = 65537  # Common choice
        d = findModInverse(e, phi)
        return (n, e), (n, d)  # public, private keys
```

Python concepts / tools covered in this chapter

★ public-key cryptography (RSA)

totient function $\phi(n)$

★ digital signatures

MITM (man-in-the-middle) attacks
hybrid cryptosystems

Chapter 24: Programming the Public Key Cipher

Implements the RSA public-key cipher. Covers converting strings into large integers (blocks) for encryption, handling block sizes to keep numbers within the RSA modulus bounds.

Explains the mathematics: encryption is c = m^e mod n, decryption is m = c^d mod n. Uses Python's built-in pow(base, exp, mod) for efficient modular exponentiation (handles huge numbers).

Covers why the public-key cipher can't be hacked with brute-force on a normal laptop — the key space is too large and the math (prime factorization) is computationally infeasible for large enough primes.

Key concepts & examples

- Modular exponentiation with pow().

```
>>> pow(47, 5, 91) # Computes 47^5 mod 91 = 7 7
```

- RSA encryption and decryption.

```
encrypted = pow(message_int, e, n) decrypted = pow(encrypted, d, n)
```

Sample code

```
def encrypt(message, pubkey):
    n, e = pubkey
    blocks = getBlocksFromText(message, blockSize)
    encrypted = []
    for block in blocks:
        encrypted.append(pow(block, e, n))
    return encrypted

def decrypt(encrypted, privkey):
    n, d = privkey
    decrypted = []
    for block in encrypted:
        decrypted.append(pow(block, d, n))
    return getTextFromBlocks(decrypted, blockSize)
```

Python concepts / tools covered in this chapter
block conversion (string <-> integer)
modular exponentiation (pow)
min() / max() functions

★ public-key encryption/decryption

Appendix: Debugging Python Code

Introduces IDLE's built-in debugger for finding and fixing bugs (exceptions and logic errors). Explains how to set breakpoints — markers that pause execution so you can inspect variable values at that point in the program.

Demonstrates debugging the reverse cipher program by setting a breakpoint, stepping through code, inspecting variables, and identifying issues. Shows how debugging is an essential skill for any programmer.

Tools/concepts covered:

IDLE debugger, breakpoints, stepping through code, variable inspection

Master Python Concepts Index (by first appearance)

Python Concept	Chapter	Description
Interactive shell / REPL	2	Python's interactive shell for one-line-at-a-time coding
Variables & assignment	2	Storing and overwriting values
String data type	3	Text manipulation (concatenation, indexing, replication)
for loop	5	Iterating over sequences
while loop	4	Loop with Boolean condition
Boolean data type	4	True/False values
Comparison operators	4	<, >, ==, != operators
if / elif / else	5	Conditional branching
in / not in operators	5	Membership testing
find() string method	5	Finding substring positions
range() function	6	Generating number sequences
f-string/.format()	6	String formatting and printing
Functions (def / return)	7	Reusable code blocks with parameters
List data type	7	Ordered, mutable sequence
Tuple data type	14	Ordered, immutable sequence
Dictionary data type	11	Key-value mapping
split() method	11	Breaking strings into word lists
strip() method	12	Removing whitespace
Multiline strings	12	Triple-quote string literals
Modulo operator (%)	13	Modular arithmetic
GCD & Euclid's algorithm	13	Greatest common divisor
Modular inverse	13	findModInverse() using extended Euclidean
Multiple assignment	13	a, b = b, a % b swapping
Exponent operator (**)	15	Power calculations
continue statement	15	Skip to next iteration
isupper()/islower()	16	Case detection for substitution
sort() method	16	Sorting lists
Word patterns	17	Letter position pattern matching
Regular expressions (re)	17	Pattern matching in text
list-append-join	18	Building strings efficiently
random module	9	Random numbers, shuffle, seed
copy.deepcopy()	9	Independent list copies
file I/O (open/read/write)	10	Reading and writing files
os.path.exists()	10	File existence checking
time module	10	time.time() for timing
set() function	20	Deduplication of collections
extend() method	20	Appending multiple items to lists

Prime numbers	22	Trial division & Rabin-Miller test
Public-key cryptography	23	RSA encryption/decryption
Digital signatures	23	Private-key signing
pow(base, exp, mod)	24	Efficient modular exponentiation

Final Notes

Cracking Codes with Python (No Starch Press, 2018) by Al Sweigart serves a dual purpose: it teaches Python programming fundamentals while simultaneously building and breaking classical ciphers. The book follows a deliberate pattern — for each cipher, first a program is written to implement it for encryption/decryption, then a program is written to hack (break) it. This hands-on approach ensures readers learn both how ciphers work and how their weaknesses can be exploited.

The ciphers covered are historical/classical — Caesar, transposition, affine, simple substitution, Vigenère, and one-time pad — plus a practical introduction to RSA public-key cryptography. These are not used for real-world security (as the author explicitly notes), but they provide an excellent foundation for understanding cryptographic principles and programming concepts.

The Python concepts taught progress naturally from variables, strings, loops, and conditionals to more advanced topics like dictionaries, lists, tuples, regular expressions, file I/O, modular arithmetic, prime number testing, and modular exponentiation. The companion website at nostarch.com/crackingcodes/ provides downloadable source code and resources.